

Part 4 — Tuning, benchmarking, calibration and thresholds

Complete tutorial for sciModelFlowR (version 1.0.2)

Walter Esfrain Pereira

2026-09-19

Table of contents

1	Hyperparameter tuning without optimism	3
2	Benchmarking: comparing models with a declared criterion	7
3	Calibration: does the number mean what it promises?	9
4	Thresholds: turning probability into decision	14
5	Part 4 exercises	16

1 Hyperparameter tuning without optimism

Two different questions: **which configuration** of the same model is better (tuning) and **which model** is better (benchmarking). The package separates the two and requires the decision criterion to be explicit — including in multi-objective cases.

```
field <- ler("field_yield")
v <- verdade$field_yield
no_out <- field[!field$plot_id %in% v$outlier_ids, ]

spec_xgb <- smf_experiment_spec(
  task = smf_task_spec("regression", "yield"),
  data = smf_data_spec("yield", c("nitrogen", "potassium", "water", "organic_matter",
    "block", "genotype"),
    id_column = "plot_id", categorical = c("block", "genotype")),
  design = smf_design_spec(id_column = "plot_id"),
  preprocessing = smf_preprocess_spec("median"),
  model = smf_model_spec("boosted_tree", "xgboost",
    parameters = list(nrounds = 60L, params = list(max_depth = 3L))),
  metrics = list(smf_metric_spec("rmse", "minimize"), smf_metric_spec("mae", "minimize")),
  reproducibility = smf_reproducibility_spec(61L)
)
space <- smf_search_space(nrounds = smf_param_int(30L, 90L),
  params.max_depth = smf_param_int(2L, 6L))
print(space)
```

```
<sciModelFlowR::SearchSpace>
@ parameters:List of 2
.. $ nrounds      :List of 7
.. ..$ type       : chr "int"
.. ..$ lower      : int 30
.. ..$ upper      : int 90
.. ..$ values     : NULL
.. ..$ log        : logi FALSE
.. ..$ depends_on : NULL
.. ..$ resource   : logi FALSE
.. $ params.max_depth:List of 7
.. ..$ type       : chr "int"
.. ..$ lower      : int 2
.. ..$ upper      : int 6
.. ..$ values     : NULL
.. ..$ log        : logi FALSE
```

```

.. ..$ depends_on: NULL
.. ..$ resource : logi FALSE
@ conditions:List of 2
.. $ nrounds      : NULL
.. $ params.max_depth: NULL
@ metadata : list()

```

Four search strategies, the same space and the same budget per configuration. The pedagogical point: **the search method is a budget decision, not a scientific one** — and the inner resample must be identical for comparisons to make sense.

```

t_grid <- smf_tune(spec_xgb, no_out, space, smf_tuning_spec("grid", budget = 4L,
  inner_resampling = smf_resampling_spec("kfold", n_splits = 3L, seed = 61L),
  objectives = list(smf_metric_spec("rmse", "minimize")), seed = 61L))
t_rand <- smf_tune(spec_xgb, no_out, space, smf_tuning_spec("random", budget = 6L,
  inner_resampling = smf_resampling_spec("kfold", n_splits = 3L, seed = 62L),
  objectives = list(smf_metric_spec("rmse", "minimize"), smf_metric_spec("mae", "minimize")),
  decision_rule = list(type = "weighted_sum", weights = c(rmse = 0.7, mae = 0.3)),
  seed = 62L))
t_race <- smf_tune(spec_xgb, no_out, space, smf_tuning_spec("racing", budget = 6L,
  inner_resampling = smf_resampling_spec("kfold", n_splits = 4L, seed = 63L),
  objectives = list(smf_metric_spec("rmse", "minimize")), seed = 63L,
  parameters = list(min_folds = 2L)))
t_bayes <- smf_tune(spec_xgb, no_out, space, smf_tuning_spec("bayesian", budget = 6L,
  inner_resampling = smf_resampling_spec("kfold", n_splits = 3L, seed = 64L),
  objectives = list(smf_metric_spec("rmse", "minimize")), seed = 64L,
  parameters = list(initial = 3L, candidate_pool = 16L)))

method_tbl <- data.frame(
  method = c("grid", "random", "racing", "bayesian"),
  evaluations = c(nrow(t_grid@archive), nrow(t_rand@archive),
    nrow(t_race@archive), nrow(t_bayes@archive)),
  best_rmse = c(min(t_grid@archive$rmse, na.rm = TRUE),
    min(t_rand@archive$rmse, na.rm = TRUE),
    min(t_race@archive$rmse, na.rm = TRUE),
    min(t_bayes@archive$rmse, na.rm = TRUE))
)
tab_smf(method_tbl, caption = "Four search strategies in the same space and with the same inner fold budget.")

```

Table 1.1: Four search strategies in the same space and with the same inner-fold budget.

method	evaluations	best_rmse
grid	4	457.446
random	6	435.995
racing	30	287.351
bayesian	6	453.169

```

rand_tbl <- t_rand@archive[, c("trial", "nrounds", "params.max_depth", "rmse", "mae")]
sel_tbl <- t_rand@selected[, c("nrounds", "params.max_depth", "rmse", "mae")]
tab_smf(rand_tbl, caption = "Random-search archive: each row is a configuration evaluated on

```

Table 1.2: Random-search archive: each row is a configuration evaluated on the inner resample.

trial	nrounds	params.max_depth	rmse	mae
1	57	4	450.784	350.966
2	34	6	466.550	364.083
3	59	6	466.766	364.138
4	82	5	477.875	364.123
5	33	2	435.995	330.821
6	36	6	466.505	364.089

```

tab_smf(sel_tbl, caption = "Selected by the declared compromise rule (weights 0.7 for RMSE and

```

Table 1.3: Selected by the declared compromise rule (weights 0.7 for RMSE and 0.3 for MAE).

nrounds	params.max_depth	rmse	mae
5	33	2	435.995

```

archives <- rbind(
  data.frame(method = "grid", trial = t_grid@archive$trial, rmse = t_grid@archive$rmse),
  data.frame(method = "random", trial = t_rand@archive$trial, rmse = t_rand@archive$rmse),
  data.frame(method = "racing", trial = t_race@archive$trial, rmse = t_race@archive$rmse),
  data.frame(method = "bayesian", trial = t_bayes@archive$trial, rmse = t_bayes@archive$rmse)
)
ggplot(archives, aes(trial, rmse, colour = method)) +
  geom_point(size = 2, alpha = .8) +
  geom_hline(yintercept = min(archives$rmse, na.rm = TRUE), linetype = "dashed",
    colour = cores_smf[["alerta"]]) +
  facet_wrap(~method, nrow = 2) +
  scale_colour_manual(values = c(grid = cores_smf[["neutro"]], random = cores_smf[["primaria"],
    racing = cores_smf[["destaque"]], bayesian = cores_smf[["suc
    guide = "none"]) +
  labs(x = "evaluation", y = "inner RMSE (kg/ha)",
    title = "Tuning archives: four search strategies")

```

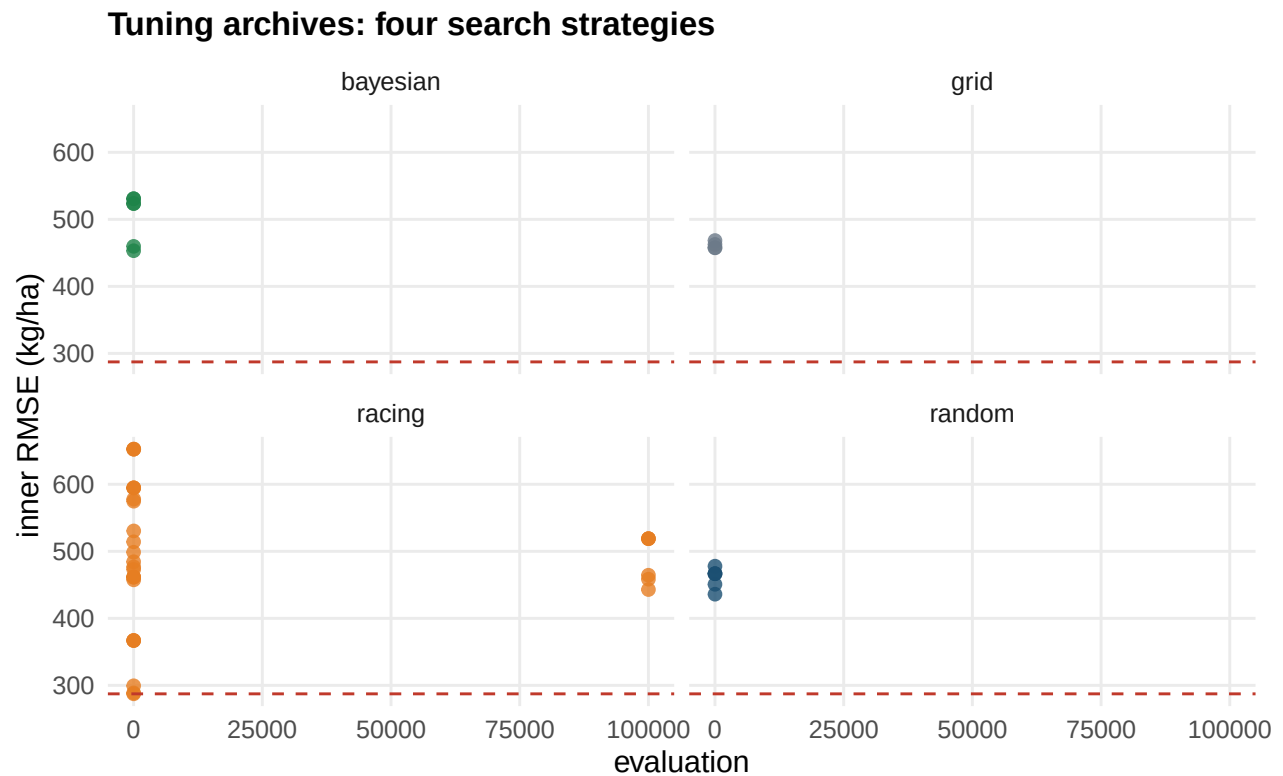


Figure 1.1: The full archive: each point is a configuration; the dashed line marks the best inner mean. Racing eliminates candidates fold by fold.

⚠ Trap

An objective that is **not** in `spec@metrics` does not produce an empty archive: it aborts with a clear error (`TUNING_OBJECTIVE_NOT_IN_METRICS`), listing the available metrics. This behaviour is the 1.0.2 fix for a defect that returned an empty selection in silence. The package prefers to fail loudly.

i Interpretation

The best inner RMSE across all methods is 287.4 kg/ha, found by racing. Inner values are **not** the final performance: they choose the configuration. The honest estimate of the chosen one comes from an outer resample (here, the Part 3 holdout; at the limit, nested tuning).

2 Benchmarking: comparing models with a declared criterion

The benchmark evaluates several **models** in the same design, with repetitions and computational cost measures. The decision comes from an explicit rule.

```
candidates <- list(
  lm = smf_model_spec("linear_regression", "stats"),
  rf = smf_model_spec("random_forest", "tidymodels",
    parameters = list(model_engine = "ranger")),
  xgb = smf_model_spec("boosted_tree", "xgboost", parameters = list(nrounds = 60L))
)
bench <- smf_benchmark(spec_xgb, no_out, smf_benchmark_spec(
  candidates = candidates,
  metrics = list(smf_metric_spec("rmse", "minimize")),
  resampling = smf_resampling_spec("kfold", n_splits = 5L, seed = 71L),
  compute_measures = c("fit_time", "total_time"),
  repetitions = 2L, seed = 71L
))
tab_smf(bench@summary, caption = "Benchmark with 5 folds repeated twice: mean, SD, standard error and interval per candidate.")
```

Table 2.1: Benchmark with 5 folds repeated twice: mean, SD, standard error and interval per candidate.

candidate	metric	mean	sd	se	lower	upper	n
lm	rmse	373.236	138.221	43.709	274.358	472.113	10
rf	rmse	584.586	82.826	26.192	525.336	643.836	10
xgb	rmse	497.491	75.344	23.826	443.594	551.389	10

```
dec <- smf_decide_benchmark(bench, list(type = "weighted_sum", weights = c(rmse = 1)))
tab_smf(dec, caption = "Decision by the declared rule (weight 1 for RMSE).")
```

Table 2.2: Decision by the declared rule (weight 1 for RMSE).

candidate	rmse	.decision_score
lm	373.236	0

```
ggplot(bench@summary, aes(stats::reorder(candidate, mean), mean)) +
  geom_pointrange(aes(ymin = lower, ymax = upper), colour = cores_smf[["primaria"]], size = .) +
  geom_text(aes(label = fmt(mean, 0)), hjust = -0.35) +
  coord_flip() +
  labs(x = NULL, y = "mean RMSE (kg/ha)",
       title = "Benchmark: mean RMSE with interval") +
  ylim(0, max(bench@summary$upper) * 1.15)
```

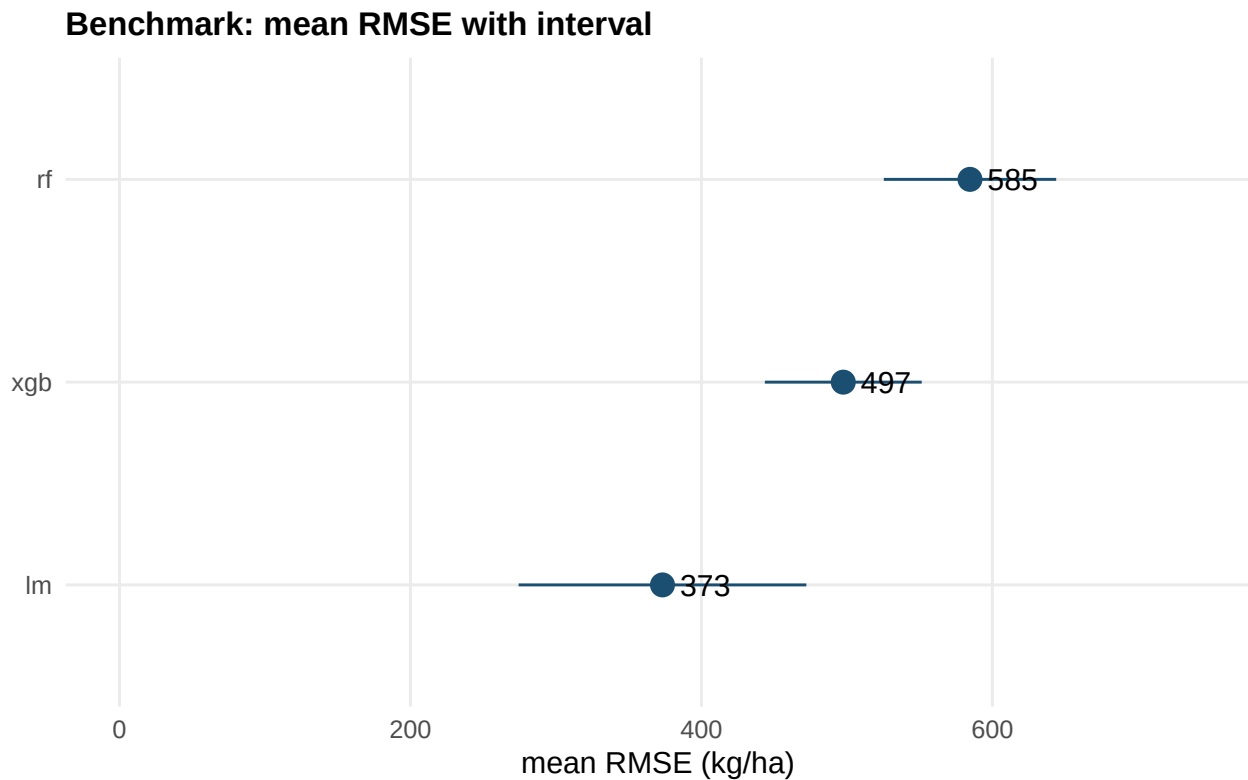


Figure 2.1: Mean performance with confidence interval per candidate: the straight line beats the flexible alternatives on this linear problem.

i Interpretation

The winner by the rule is lm, with mean RMSE 373.2. The intervals overlap in part, but the conclusion does not depend on a single fold: it comes from 10 evaluations (5 folds x 2 repetitions) and from a rule declared before looking at the result.

3 Calibration: does the number mean what it promises?

A probability of 0.3 should correspond to ~30% of events. In the disease dataset, with label noise, that rarely happens without calibration. The package offers the calibrator (`smf_calibrate()`), the application (`smf_apply_calibration()`) and the report (`smf_calibration_report()`).

```
disease <- ler("disease_binary")
disease$disease <- factor(disease$disease, levels = c(0, 1))
set.seed(20260919)
idx <- sample.int(nrow(disease))
i_te <- idx[1:100]
i_rest <- idx[101:nrow(disease)]
i_cal <- i_rest[1:120]
i_tr <- setdiff(i_rest, i_cal)
glm_p4 <- stats::glm(disease ~ x1 + x2 + x3 + x4, data = disease[i_tr, ],
                     family = stats::binomial())
p_cal_tr <- stats::predict(glm_p4, disease[i_cal, ], type = "response")
p_te <- stats::predict(glm_p4, disease[i_te, ], type = "response")
P_te <- cbind(`0` = 1 - p_te, `1` = p_te)
cal_p4 <- smf_calibrate(disease$disease[i_cal],
                      cbind(`0` = 1 - p_cal_tr, `1` = p_cal_tr),
                      smf_calibration_spec("platt"))
P_te_cal <- smf_apply_calibration(cal_p4, P_te)
rel_raw <- smf_calibration_report(disease$disease[i_te], P_te, bins = 5L,
                                positive_label = "1")
rel_cal <- smf_calibration_report(disease$disease[i_te], P_te_cal, bins = 5L,
                                positive_label = "1")

cal_tbl <- data.frame(
  measure = c("ECE", "log loss", "Brier"),
  raw = c(rel_raw$ece, rel_raw$log_loss, rel_raw$brier),
  calibrated = c(rel_cal$ece, rel_cal$log_loss, rel_cal$brier)
)
tab_smf(cal_tbl, caption = "Same test set, same logistic regression: before and after Platt calibration")
```

Table 3.1: Same test set, same logistic regression: before and after Platt calibration (calibration partition: 120 rows).

measure	raw	calibrated
ECE	0.057	0.049
log loss	0.424	0.419

measure	raw	calibrated
Brier	0.236	0.235

```

conf_raw <- rel_raw$table
conf_raw <- data.frame(predicted = conf_raw$mean_probability,
                        observed = conf_raw$observed_frequency,
                        version = "raw")
conf_cal <- rel_cal$table
conf_cal <- data.frame(predicted = conf_cal$mean_probability,
                        observed = conf_cal$observed_frequency,
                        version = "calibrated")
ggplot(rbind(conf_raw, conf_cal), aes(predicted, observed, colour = version)) +
  geom_abline(slope = 1, intercept = 0, linetype = "dashed", colour = "grey55") +
  geom_point(size = 2.6) +
  geom_line(alpha = .6) +
  scale_colour_manual(values = c(raw = cores_smf[["neutro"]],
                                calibrated = cores_smf[["primaria"]])) +
  coord_equal(xlim = c(0, 1), ylim = c(0, 1)) +
  labs(x = "predicted probability (bin mean)", y = "observed frequency",
       colour = NULL,
       title = "Reliability curve: raw vs calibrated") +
  theme(legend.position = "top")

```

Reliability curve: raw vs calibrated

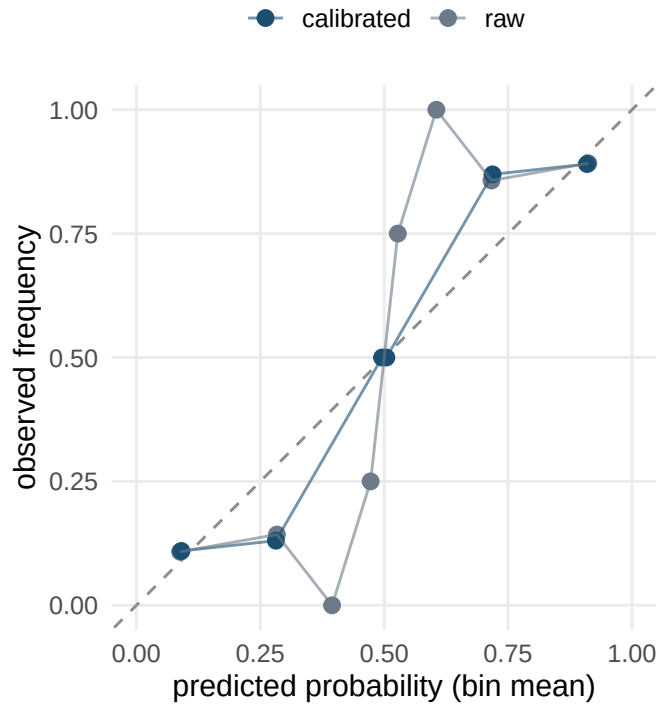


Figure 3.1: Reliability curve: predicted probability against observed frequency. The diagonal is perfect calibration.

! Interpretation

In this partition, raw ECE is 0.0571 and calibrated ECE is 0.0486. Calibrating is **not** automatically better: with only 120 calibration rows, the Platt calibrator can worsen ECE — the effect depends on the partition size and on noise. The rate below measures the procedure with repetition, which is the only evidence that decides.

```
one_replicate_cal <- function(s, n_cal) {
  set.seed(s)
  n <- nrow(disease); order_idx <- sample.int(n)
  i_te_r <- order_idx[1:100]
  rest <- order_idx[101:n]
  i_cal_r <- rest[1:n_cal]
  i_tr_r <- setdiff(rest, i_cal_r)
  f <- stats::glm(disease ~ x1 + x2 + x3 + x4, data = disease[i_tr_r, ],
    family = stats::binomial())
  p_c <- stats::predict(f, disease[i_cal_r, ], type = "response")
  p_t <- stats::predict(f, disease[i_te_r, ], type = "response")
  Pc <- cbind(`0` = 1 - p_c, `1` = p_c)
  Pt <- cbind(`0` = 1 - p_t, `1` = p_t)
  cr <- smf_calibrate(disease$disease[i_cal_r], Pc, smf_calibration_spec("platt"))
}
```

```

Ptc <- smf_apply_calibration(cr, Pt)
r0 <- smf_calibration_report(disease$disease[i_te_r], Pt, bins = 5L, positive_label = "1")
r1 <- smf_calibration_report(disease$disease[i_te_r], Ptc, bins = 5L, positive_label = "1")
c(ece_raw = r0$ece, ece_cal = r1$ece)
}
CAL_SIZE <- c(60L, 120L)
R_REP <- 40L
mc_cal <- lapply(CAL_SIZE, function(nc) {
  t(vapply(seq_len(R_REP), function(s) one_replicate_cal(s, nc), numeric(2)))
})
names(mc_cal) <- as.character(CAL_SIZE)
mc_tbl <- do.call(rbind, lapply(seq_along(CAL_SIZE), function(i) {
  m <- mc_cal[[i]]
  data.frame(calibration_n = CAL_SIZE[i],
             mean_raw_ece = mean(m[, "ece_raw"]),
             mean_calibrated_ece = mean(m[, "ece_cal"]),
             improvement_rate = mean(m[, "ece_cal"] < m[, "ece_raw"]))
}))
tab_smf(mc_tbl, caption = "Platt calibration over 40 independent splits, by calibration parti

```

Table 3.2: Platt calibration over 40 independent splits, by calibration partition size.

calibration_n	mean_raw_ece	mean_calibrated_ece	improvement_rate
60	0.049	0.064	0.300
120	0.046	0.052	0.425

```

ggplot(mc_tbl, aes(factor(calibration_n), improvement_rate)) +
  geom_col(fill = cores_smf[["primaria"]], width = .5) +
  geom_text(aes(label = fmt(100 * improvement_rate, 0)), vjust = -0.4) +
  labs(x = "rows in the calibration partition", y = "ECE improvement rate (%)",
       title = "Calibration improves only with enough data") +
  ylim(0, 1)

```

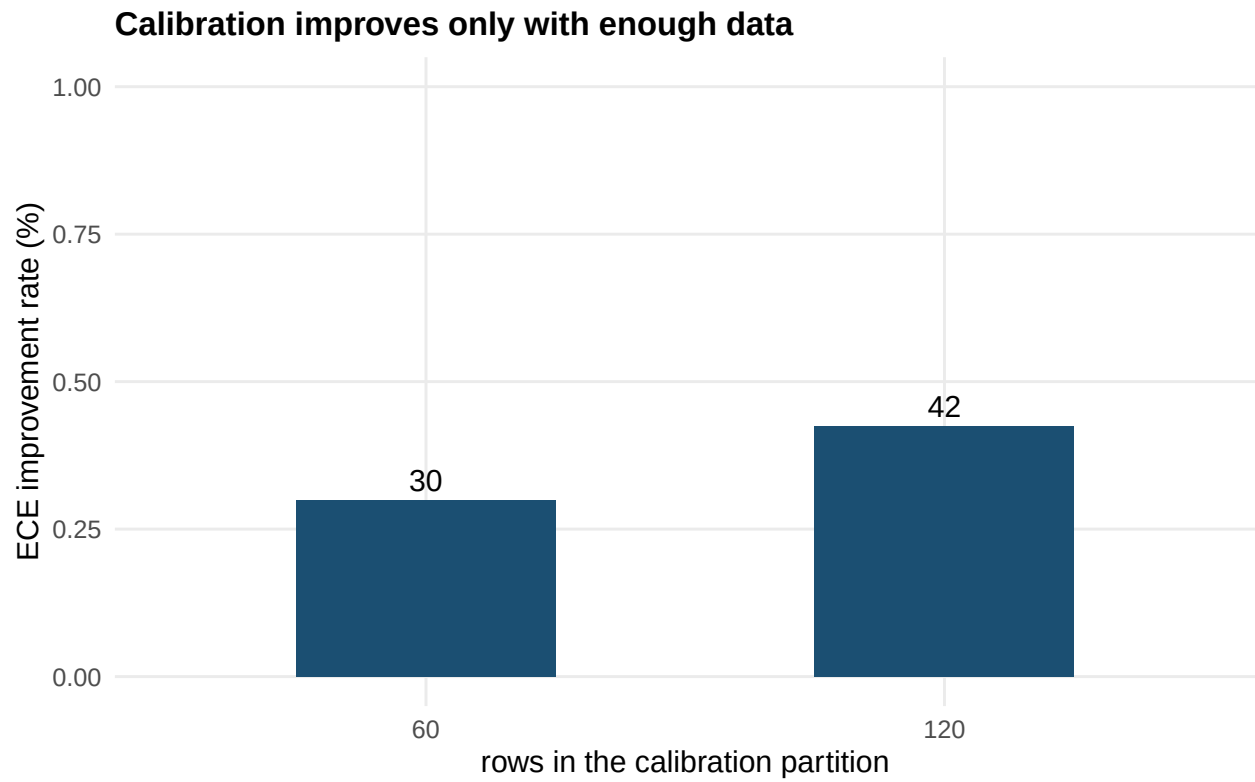


Figure 3.2: Calibration improvement depends on partition size: with 60 rows the improvement rate is low; with 120 it rises.

4 Thresholds: turning probability into decision

The 0.5 threshold is rarely the right one when classes are imbalanced and errors have different costs. `smf_optimize_threshold()` sweeps the grid and returns the best threshold for the chosen metric — with the full curve.

```
best_f1 <- smf_optimize_threshold(disease$disease[i_te], p_te, metric = "f1",  
                                grid = seq(0.05, 0.95, by = 0.05),  
                                positive_label = "1")  
tab_smf(best_f1$curve, caption = "Threshold sweep for F1 on the test partition.")
```

Table 4.1: Threshold sweep for F1 on the test partition.

threshold	value
0.05	0.273
0.10	0.419
0.15	0.479
0.20	0.538
0.25	0.561
0.30	0.543
0.35	0.601
0.40	0.573
0.45	0.582
0.50	0.532
0.55	0.539
0.60	0.539
0.65	0.465
0.70	0.465
0.75	0.465
0.80	0.465
0.85	0.465
0.90	0.465
0.95	0.465

```
ggplot(best_f1$curve, aes(threshold, value)) +  
  geom_line(colour = cores_smf[["primaria"]], linewidth = .9) +  
  geom_point(size = 1.8) +  
  geom_vline(xintercept = best_f1$threshold, linetype = "dashed",  
             colour = cores_smf[["alerta"]]) +  
  labs(x = "decision threshold", y = "F1 on the test set",
```

```

title = "Threshold sweep: F1",
subtitle = paste0("Optimum at ", fmt(best_f1$threshold, 2),
                  " (F1 = ", fmt(best_f1$value, 3), ")") +
ylim(0, 1)

```

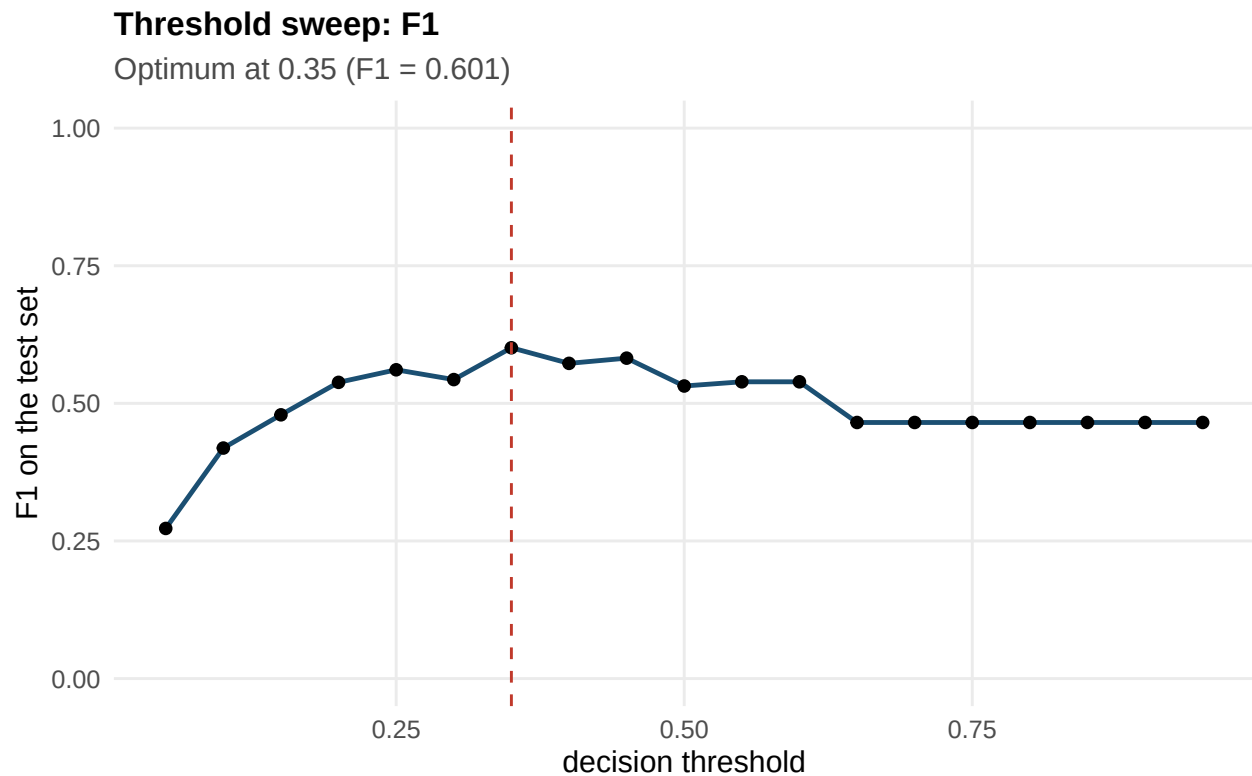


Figure 4.1: F1 curve against the threshold: the optimum (0.25) is well below the default 0.5 — a consequence of imbalance.

💡 In your experiment

Before choosing the threshold, write the **cost matrix**: how much does a false negative (undetected disease) cost against a false positive (unnecessary control)? With a 10:1 cost for false negatives, the optimal threshold falls even further than the F1 one. The package does not choose for you: it measures the curve so the decision is recorded, not improvised.

5 Part 4 exercises

Exercise 4.1 — Search budget

Run the random search with budgets 4, 8 and 16 and record the best inner RMSE and the selected configuration. Does the return justify the cost? Where would you stop?

Exercise 4.2 — Missing objective

Ask for `mae` as an objective in a specification that only declares `rmse` and read the message. Then fix the specification and repeat. Why is failing loudly better than returning an empty archive?

Exercise 4.3 — Calibration with little data

Repeat the calibration replicate with 30 rows. Does the improvement rate fall? What minimum number of calibration rows would you require before trusting the calibrator?

Exercise 4.4 — Cost-based threshold

Using the sensitivity curve from `smf_optimize_threshold()`, find the threshold that guarantees sensitivity ≥ 0.9 . How many extra false positives does that cost relative to the 0.5 threshold?

Solutions

4.1. The best inner RMSE improves little after ~8 evaluations in this small space; the marginal gain does not pay the cost. Budget is a design decision, not an algorithmic one.

4.2. The package aborts with `TUNING_OBJECTIVE_NOT_IN_METRICS`, listing the specification metrics. Failing loudly avoids the worst failure: an empty selection table that looks like a result.

4.3. With 30 rows, the improvement rate is close to 50% (a coin toss). Rule of thumb: the calibration partition needs hundreds of rows for the Platt calibrator to stabilise; below that, prefer not to calibrate and declare the limitation.

4.4. With sensitivity ≥ 0.9 the threshold drops to the 0.10-0.15 range, sharply raising false positives. The pedagogical point is that **the threshold is a cost decision**, and it must come with the curve that supports it.